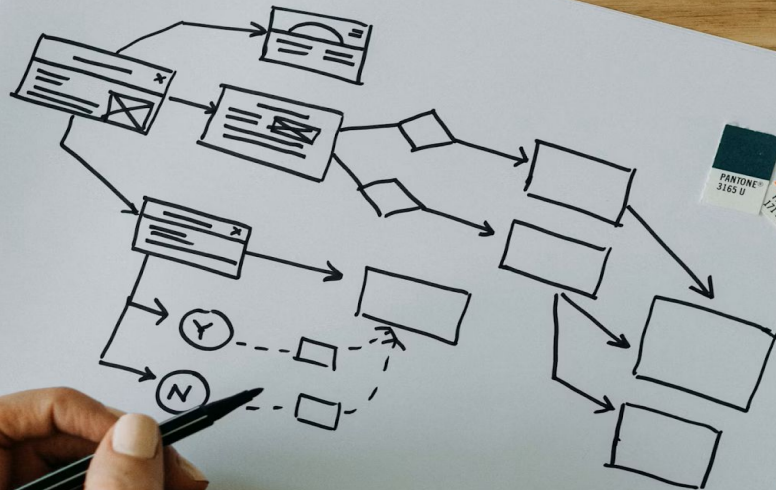


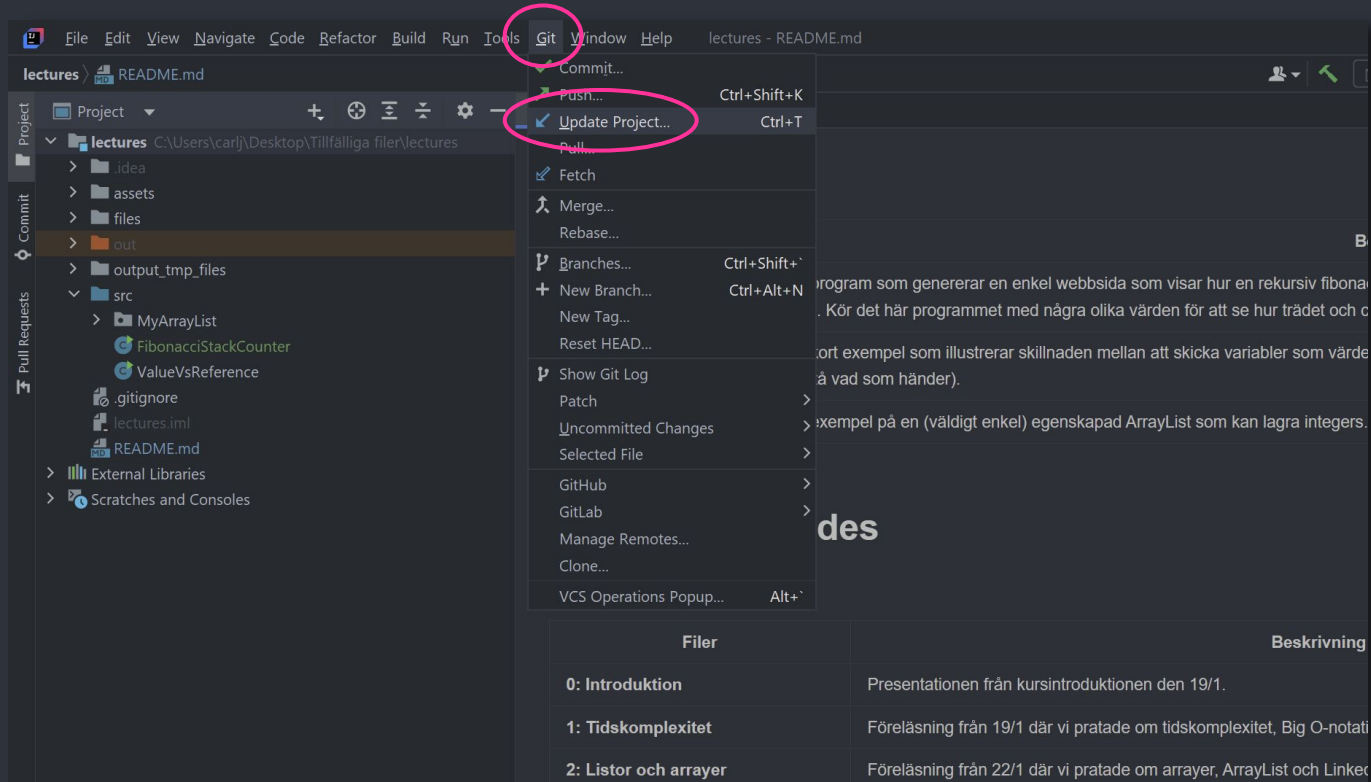
Algoritmer och Datastrukturer

Föreläsning 9



Projektstart

<https://github.com/carljohanj/algo>



Programråd

- Martin och Thomas vill ha folk till programrådet: om du läser Programvaruteknik och är intresserad får du gärna kontakta dem (eller säga till mig så delegerar jag vidare!)
- Bra merit och ni hjälper till att förbättra er egen utbildning
- Bara ca två möten per termin
- Ni får fika!



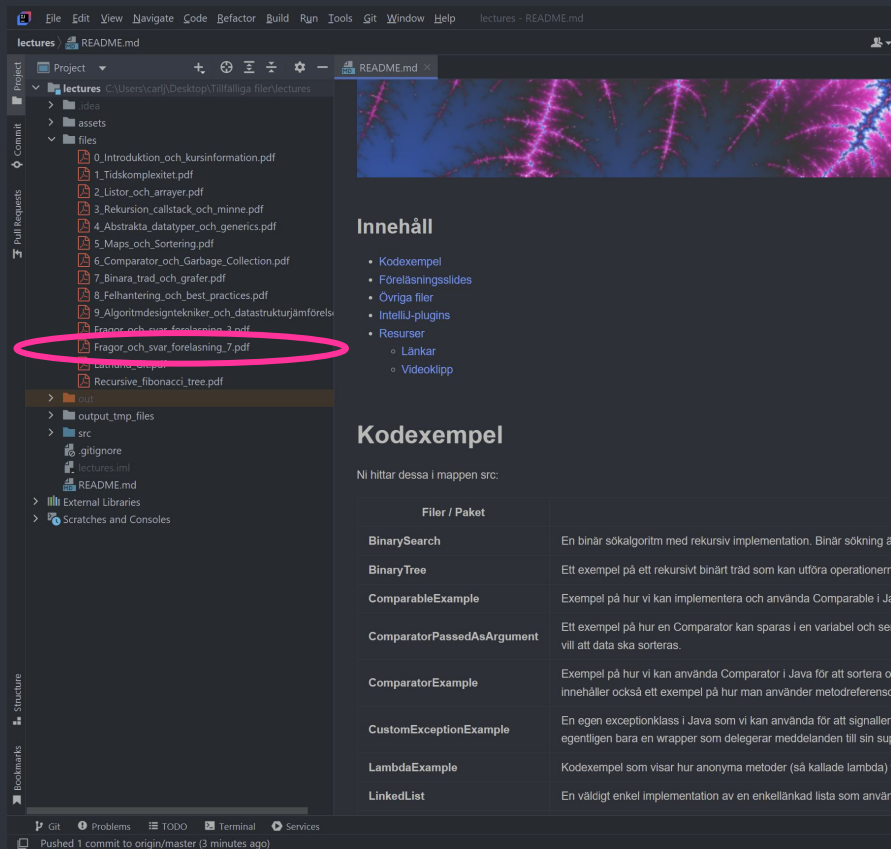
Resten av kursen

- **På måndag (23 februari) kör vi en heldag med Test-Driven Development** där vi lär oss skriva testklasser och hur man bygger upp en algoritm från grunden
- **Onsdagen den 11 mars** kommer vi ha **slutseminarium för projektet:** det här tillfället står inte i schemat eftersom vi inte kan schemalägga de små grupprummen på institutionen, men vi kommer skicka ut information om det när det närmar sig så att alla vet var de ska vara
- **Torsdagen den 12 mars** har vi ett **repetitionstillfälle där vi kommer gå genom gamla tentor** tillsammans: bra plugg inför tentamen!
- Kom på labbpassen och ställ frågor och jobba med uppgifter, projekt, repetera vad vi gjort hittills... vänta inte till sista dagarna med att repetera utan börja redan nu

Frågor från föreläsning 7

- 1) Vad är en rekursiv datastruktur för någonting?
- 2) Vilken tidskomplexitet har en sökning i ett rödsvart träd? Varför?
- 3) Vad är risken med obalanserade binära träd?
- 4) Ge exempel på några datastrukturer i Java som använder sig av balanserade träd.
- 5) Vad är en graf?
- 6) Ge några exempel på användningsområden för grafer.
- 7) Vad är garbage collection för någonting? Varför används det i Java?
- 8) Vad är några av fördelarna med en enhanced for loop?
- 9) Vad är några av nackdelarna?
- 10) Vilka är de tre vanliga sätten att traversera ett binärt sökträd? Vad är skillnaden mellan dem?
- 11) Vad är Dijkstras algoritm för någonting? Hur fungerar den?
- 12) Vad är undantag och hur kan man hantera dem?
- 13) Vad är stack unwinding (stackupplindning) för någonting?
- 14) Vad är skillnaden mellan checked och unchecked exceptions (märkta och omärkta undantag)?
- 15) Vad är tvåstegsinitialisering och varför ska man undvika det?
- 16) Vad är en girig algoritm? Hur fungerar den?

Svaren finns i repot



The screenshot shows an IDE interface with a file explorer on the left and a content pane on the right. The file explorer shows a project structure with a folder named 'lectures' containing several PDF files. The file 'Fragor och svar forelasning 7.pdf' is circled in pink. The content pane on the right has a header image and two sections: 'Innehåll' (Table of Contents) and 'Kodexempel' (Code Examples).

Innehåll

- Kodexempel
- Föreläsningsslides
- Övriga filer
- IntelliJ-plugins
- Resurser
 - Länkar
 - Videoklipp

Kodexempel

Ni hittar dessa i mappen src:

Filer / Paket	
BinarySearch	En binär sökalgoritm med rekursiv implementation. Binär sökning är
BinaryTree	Ett exempel på ett rekursivt binärt träd som kan utföra operationer:
ComparableExample	Exempel på hur vi kan implementera och använda Comparable i Ja
ComparatorPassedAsArgument	Ett exempel på hur en Comparator kan sparas i en variabel och sen vill att data ska sorteras.
ComparatorExample	Exempel på hur vi kan använda Comparator i Java för att sortera ot innehåller också ett exempel på hur man använder metodreferensor
CustomExceptionExample	En egen exceptionklass i Java som vi kan använda för att signalisera egentligen bara en wrapper som delegerar meddelanden till sin sup
LambdaExample	Kodexempel som visar hur anonyma metoder (så kallade lambda) f
LinkedList	En väldigt enkel implementation av en enkellänkad lista som anvä

Dugga 1!

Dugga 1: Tidskomplexitet

3

Fyll i den tomma rutan 7,5 poäng

n	tid
2	4
4	34
8	260
16	2 040
32	16 406

Metoden vars tidstestning visas i tabellen ovan har troligen följande tidskomplexitet:

välj ditt svar ...



Om tiden dubblas när datamängden dubblas: $O(n)$
Om tiden fyrdubblas när datamängden dubblas: $O(n^2)$
Om tiden åttadubblas när datamängden dubblas: $O(n^3)$

Svar: $O(n^3)$

Datamängden dubblas t.ex. Mellan 4 och 8, 8 och 16, och 16 och 32 – vad händer med tiden?

Dugga 1: vad returnerar metoden?

5

Fyll i den tomma rutan 15 poäng

```
public boolean method(int[] a, int[] b) {  
    if (a.length != b.length) {  
        return false;  
    }  
    for (int i = 0; i < a.length; i++) {  
        if (a[i] != b[i]) {  
            return false;  
        }  
    }  
    return true;  
}
```

Om metoden anropas med a = [1, 2, 2, 1] och b = [1, 2, 2, 1] så kommer den returnera

skriv in ditt svar ... **true**

Om metoden anropas med a = [1, 2, 2, 1] och b = [2, 1, 1, 2] så kommer den returnera

skriv in ditt svar ... **false**

Metoden har följande tidskomplexitet i bästa fall:

välj ditt svar ...

$O(1)$

Metoden har följande tidskomplexitet i värsta fall:

välj ditt svar ...

$O(n)$

Vilken datastruktur?

7

Flera val 10 poäng



Du skall utveckla en applikation som behöver lagra mycket data. När applikationen används kommer applikationen *ibland* behöva lista alla poster, *ofta* lägga till poster först eller sist, *aldrig* ta bort enskilda poster och *sällan* söka efter enskilda poster. Vilken av följande datastrukturer är mest lämplig för att lagra data i applikationen?

- ☐ Array
- ☐ Länkad lista
- ☐ Stack
- ☐ Kö

Krav:

- * Ofta lägga till först/sist
- * Aldrig ta bort
- * Sällan söka

Jämförelse av datastrukturer

	Lista Samtliga	Lägga till	Ta bort	Söka	Intervall
Array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(k)$
ArrayList	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(k)$
LinkedList	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$(n+k)$
TreeMap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
HashMap	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(k)$

Vilken sorteringsalgoritm?

7

Flera val 10 poäng



Du skall utveckla en applikation som behöver lagra mycket data. När applikationen används kommer applikationen *ibland* behöva lista alla poster, *ofta* lägga till poster först eller sist, *aldrig* ta bort enskilda poster och *sällan* söka efter enskilda poster. Vilken av följande datastrukturer är mest lämplig för att lagra data i applikationen?

- ☐ Array
- ☐ Länkad lista
- ☐ Stack
- ☐ Kö

Krav:

- * Ofta lägga till först/sist
- * Aldrig ta bort
- * Sällan söka

Svar: Länkad lista! Den är dålig på sökning och insättning i mitten om vi inte vet var noden är, men vi kan sätta in i båda ändarna på konstant tid. Att lista alla är $O(n)$ precis som en array så där är ingen skillnad

Vilken sorteringsalgoritm?

10

0/10 poäng Flera val

Du skall utveckla en applikation som behöver sortera listor med data. Listorna är aldrig sorterade och du vill använda en sorteringsalgoritm som är *in-place* och *effektiv*. Vilken sorteringsalgoritm använder du?

- ☐ Bubble Sort
- ☐ Selection Sort
- ☐ Insertion Sort
- ☐ Merge Sort
- ☐ Quick Sort

Jämförelse, olika sorteringsalgoritmer

Algoritm	Tidskomplexitet (Bäst/Sämst)	In-Place	Stabil	Används
QuickSort	$O(n \cdot \log n)$ / $O(n^2)$	Ja	Nej	Stora dataset som behöver in-place-sortering
MergeSort	$O(n \cdot \log n)$	Nej	Ja	När stabilitet är viktigt
InsertionSort	$O(n)$ / $O(n^2)$	Ja	Ja	Små eller mestadels sorterade samlingar
BubbleSort	$O(n^2)$	Ja	Ja	Helst inte, men ok för små dataset
SelectionSort	$O(n^2)$	Ja	Nej	Används sällan men föredras över BubbleSort pga färre swaps

- Giltiga algoritmer
- Används främst i lärosyfte

Vilken sorteringsalgorithm?

10

0/10 poäng Flera val

Du skall utveckla en applikation som behöver sortera listor med data. Listorna är aldrig sorterade och du vill använda en sorteringsalgorithm som är *in-place* och *effektiv*. Vilken sorteringsalgorithm använder du?

- ☐ Bubble Sort
- ☐ Selection Sort
- ☐ Insertion Sort
- ☐ Merge Sort
- ☐ Quick Sort

Svar: Bubble Sort och Selection Sort är inte effektiva alls så de utgår.

Insertion Sort är stabil och in-place men inte effektiv.

Merge Sort är effektiv men inte in-place.

Quick Sort är effektiv och in-place. Den är inte stabil, men det var inte ett krav i det här fallet. **Svaret är alltså Quick Sort.**

Algoritmdesigntechnik

8

0/10 poäng Fyll i den tomma rutan

```
public int method(int[] a, int b) {  
    Arrays.sort(a);  
    return Arrays.binarySearch(a, b);  
}
```

Om metoden anropas med $a = [3, 1, 5]$ och $b = 3$ så kommer metoden returnera **1**

Om metoden anropas med $a = [-4, -6, -2]$ och $b = -2$ så kommer metoden returnera **2**

Metoden har följande tidskomplexitet i normalfallet: **$O(n \log n)$**

Funktionalitet i Javas API använder följande algoritmdesigntechnik:

Divide-and-conquer

Projektuppgift: väderdata från Visby flygplats mellan 1945 och 2025

- På uppgiftssidan finns det en fil med väderdata från Visby flygplats mellan 1945 och 2025, en till textfil som förklarar vad datan är för något, samt en projektmall som innehåller tre javaklasser
- Er uppgift är att implementera metoderna som finns beskrivna med JavaDoc i klassen WindDataHandler
- Ni får inte ändra i signaturerna eller i beskrivningarna för dessa metoder, men ni är fria i övrigt att lägga till så många klasser och metoder ni vill

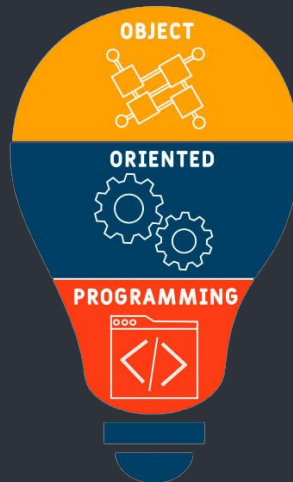


Tänk objektorienterat

- Glöm inte att ni kan skapa era egna objekt när ni jobbar med objektorientering: fundera på vad som hör samman, och var
- En bra tumregel är att **klasser är substantiv**, dvs namn på saker:

De innehåller både **data** (variabler) **och beteenden** (metoder som manipulerar datan på något vis, ex: hämtar ut datan, låter oss skriva över den, summerar saker... osv)

- **Metoder är verb** eftersom de utför en aktiv handling:
`readFromFile()`, `getName()`, `findValue()`, osv
- Vi förväntar oss en objektorienterad lösning i projektet men exakt hur den ser ut får ni själva avgöra och motivera: de tre klasser som ni fått på förhand ska dock vara med



Förstå uppgiften innan ni börjar

- All forskning som finns visar på att **ju tidigare man sätter igång och börjar koda ett projekt, desto längre tid tar det att färdigställa**
- Var inte för ivriga med att komma igång: tänk genom uppgiften ordentligt först och se till att ni faktiskt förstår den (kan behöva en dag eller två för att sjunka in ordentligt)
- Titta på datan ni ska arbeta med och läs legenden som förklarar vad den är för något
- **Tänk också noga genom valet av datastruktur** eftersom det här kommer vara svårt att ändra på i efterhand: innan ni börjar koda bör ni ha vägt för- och nackdelar mot andra strukturer vi gått genom på kursen (kolla genom Föreläsningsmaterialet för bra tips!)
- **I både rapporten och på seminariet ska ni kunna motivera** på ett tydligt vis varför ni valt den här strukturen och vad alternativen hade varit



Tidskomplexiteter

- Ni ska beskriva era tidskomplexiteter i både koden och i rapporten: kommentera gärna vad tidskomplexiteten är för varje rad så som vi gjort i de kodexempel vi gått genom på föreläsningarna
- Tanken är att ni ska skriva kod som är tidseffektiv: hellre $O(n)$ än $O(n^2)$ osv
- Det finns självklart annat som kan påverka tidskomplexiteten i era algoritmer än bara datastrukturen som används för att lagra datan, och därför är det bra att tänka genom allting en extra gång innan ni börjar koda: har jag bra uttänkta system för sökning, uthämtning, insättning, iteration, osv? Vilka är riskerna jag måste undvika?

LocalDate

//Skapa ett LocalDate-objekt utifrån en sträng som innehåller ett datum

```
LocalDate date20220101 = LocalDate.parse("2022-01-01");
```

```
LocalDate date20201231 = LocalDate.parse("2020-12-31");
```

//Lägger till eller drar ifrån ett antal dagar (ändrar inte värdet på det ursprungliga objektet)

```
LocalDate date20220102 = date20220101.plusDays(1);
```

```
LocalDate date20211222 = date20220101.minusDays(10);
```

//Några kontroller

```
boolean isTrue = date20220102.isAfter(date20220101);
```

```
boolean isFalse = date20220101.isBefore(date20201231);
```

//Skriver ut alla datum i januari 2022

```
LocalDate date20220101 = LocalDate.parse("2022-01-01");
```

```
LocalDate date20220201 = LocalDate.parse("2022-02-01");
```

```
while (date20220201.isAfter(date20220101))
```

```
{
```

```
    System.out.println(date20220101);
```

```
    date20220101 = date20220101.plusDays(1);
```

```
}
```

- **Projektet använder LocalDate** som är en inbyggd klass i Java för att hantera datum
- Ni förväntas använda dessa i projektet (ni får även lov att använda andra klasser från java.time **utöver** LocalDate om ni vill!)
- Några vanliga operationer man kan göra på datumobjekt listas till vänster
- Läs mer här (eller googla själva!):
https://www.w3schools.com/java/java_date.asp

Versionshistorik

- Projektet **måste ha en utförlig versionshistorik** med **atomära commits**: projektinlämningar som inte har utförlig versionshistorik kommer inte att accepteras
- Projekt med commits som alla skapats samma tillfälle eller samma dag kommer inte att godkännas
- Vi har **gott om Gitresurser under modulerna på kurssidan** om man behöver hjälp med versionshantering
- Det ingår i rapporten att reflektera över hur det varit att jobba med versionshantering i ett större projekt: läs gärna på om hur man gör rollbacks och arbetar med olika branches i fall ni skulle behöva använda er av dessa funktioner!
- **Git är er bästa vän** när ni arbetar med större projekt
- **I samband med att ni lämnar in projektet bjuder ni också in mig, Lovisa och Sara till era repon på github** (våra git-handles är listade på förstasidan för kursen)



Felhantering

- Tänk på att **programmet ska vara robust**: fundera över vilken slags fel som kan uppstå och försök förebygga dem
- **Datum måste fungera**: vi ska t.ex. inte kunna söka på datum före första datumet eller datum som ligger efter sista datumet i den inlästa datan
- Tänk även på att programmet ska klara de föreskrivna test-exemplen i JavaDoc-kommentarerna
- Ni har fria händer att göra som ni vill, **men programmet ska ha felhantering som inte bara är en eftertanke**: fundera på vad vi lärt oss om NullPointerExceptions, undantag, validering, osv

Rapport

- Behöver inte vara akademiskt uppbyggd, men får vara det om ni vill!
- **Ni behöver skriva den här själva:** vi kommer underkänna allt som är skrivet med AI eftersom tanken är att **ni ska resonera kring och motivera era val och ert arbete**
- Tänk på att **rapporten ska hålla en bra språklig standard** även om det här inte är en skrivkurs: har ni problem med grammatik och liknande så kontakta Språkverkstan på universitetet, be andra studenter att korrekturläsa, osv
- Det viktiga i rapporten är dock att ni på ett utförligt sätt beskriver de val ni gjort och varför
- **Det kan vara bra att läsa genom kraven på rapporten ordentligt innan ni börjar koda,** för de frågor ni ska besvara handlar om saker som är viktiga att tänka på innan ni sätter i gång: vilka alternativ finns, vad är för- och nackdelar med olika datastrukturer, vilken tidskomplexitet bör jag kunna förvänta mig? Osv



Seminarium

- Ni har två veckor på er att slutföra projektet: fredagen den 6 mars kl 21:00 är en **HÅRD DEADLINE**
- Projekt som kommer in en minut efter deadline kommer inte att accepteras: ni får då en ny chans på uppsamlingsseminariet i slutet av maj (observera att ni måste skicka in igen för att få vara med vid det tillfället)
- **Onsdagen den 11 mars kommer vi ha ett slutseminarium** där alla som skickat in i tid opponerar på en annan students projektinlämning
- **Bra övning i att läsa och förstå andras kod**, men också ett bra tillfälle att lära sig att ge och acceptera konstruktiv kritik
- **Seminariet står inte utskrivet i schemat utan kommer äga rum i små grupper i mötesrummen på institutionen:** ni kommer få personliga utskick där det står var ni ska vara och när

Seminarium

- Blir man godkänd på seminarium och projektinlämning är man godkänd på uppgiften
- Blir man godkänd på själva projektet men inte presterar tillräckligt bra på seminariet så behöver man göra om seminariet för att bli godkänd: man måste i så fall skicka in sitt projekt på nytt så att man är med och får en ny opponering i vår (men man behöver inte göra om själva projektet i övrigt om läraren tycker att det ser bra ut)
- Därför är det **viktigt att man på seminariet är väl förberedd** på att inte bara opponera utan även bemöta eventuella frågor om sitt projekt från opponenter och övriga gruppen
- Skulle man inte nå upp till godkänt på seminariet så finns det fler tillfällen! **Bli inte ledsna utan ta det som en lärandegrej** bara - målet är att ni ska utvecklas och bli bättre programmerare på den här kursen